

Q1.

```
let arr = [1, 2, 3];  
arr.unshift(0);  
arr.shift();  
console.log(arr);
```

- A) [0,1,2,3]
- B) [1,2,3]
- C) [0,2,3]
- D) [2,3]

Answer: B

Q2.

```
let arr = [1,2,3,4];  
let res = arr.slice(1,3);  
arr.splice(1,2);  
console.log(res, arr);
```

- A) [2,3] [1,4]
- B) [2,3] [1,2,3,4]
- C) [1,2] [3,4]
- D) [2,3] [1,3,4]

Answer: A

Q3.

```
let arr = [1,2,3];  
let res = arr.flatMap(x => [x, x*2]);  
console.log(res);
```

- A) [1,2,2,4,3,6]
- B) [[1,2],[2,4],[3,6]]
- C) [2,4,6]
- D) Error

Answer: A

Q4.

```
let arr = [10,20,30];  
console.log(arr.findIndex(x => x > 15));
```

- A) 0
- B) 1
- C) 2
- D) -1

Answer: B

Q5.

```
let arr = [1,2,3,4];  
let res = arr.reduce((a,b)=>a*b);  
console.log(res);
```

- A) 10
- B) 24
- C) 20
- D) 0

Answer: B

Q6.

```
let arr = [1,2,3];  
let newArr = arr.with(1, 99);  
console.log(arr, newArr);
```

- A) [1,99,3] [1,99,3]
- B) [1,2,3] [1,99,3]
- C) [1,2,3] [1,2,3]
- D) Error

Answer: B

Q7.

```
let obj = {a:1, b:2};  
let res = Object.values(obj);  
console.log(res);
```

- A) ["a","b"]
- B) [1,2]
- C) [["a",1],["b",2]]
- D) Error

Answer: B

Q8.

```
let obj = {a:1};  
let newObj = Object.assign(obj, {b:2});  
console.log(obj);
```

- A) {a:1}
- B) {b:2}
- C) {a:1,b:2}
- D) Error

Answer: C

Q9.

```
let key = "name";  
let obj = {};  
obj[key] = "Amay";  
console.log(obj.name);
```

- A) undefined
- B) Amay
- C) Error
- D) null

Answer: B

Q10.

```
let [a,,b] = [5,10,15];  
console.log(a,b);
```

- A) 5,10
- B) 5,15
- C) 10,15
- D) undefined

Answer: B

Q11.

```
let {a=10, b=20} = {b:5};  
console.log(a,b);
```

- A) 10,20
- B) undefined,5
- C) 10,5
- D) Error

Answer: C

Q12.

```
let x=1, y=2;  
[x,y] = [y,x];  
console.log(x,y);
```

- A) 1,2
- B) 2,1
- C) Error
- D) undefined

Answer: B

Q13.

```
let obj = {a:1};  
let str = JSON.stringify(obj);  
console.log(typeof str);
```

- A) object
- B) string
- C) number
- D) undefined

Answer: B

Q14.

```
let str = '{"a":5}';  
let obj = JSON.parse(str);  
console.log(obj.a);
```

- A) "5"
- B) 5
- C) undefined
- D) Error

Answer: B

Q15.

```
let el = document.createElement("p");  
el.innerText = "Hello";  
document.body.appendChild(el);
```

What happens?

- A) Nothing
- B) Paragraph added
- C) Error
- D) Replaces body

Answer: B

Q16.

```
document.getElementById("demo").innerHTML = "<b>Hi</b>";
```

What will display?

- A) Hi
- B) Hi (bold)
- C) Error
- D) Nothing

Answer: B

Q17.

```
button.addEventListener("click", e => {  
  e.preventDefault();  
});
```

What does it do?

- A) Stops bubbling
- B) Prevents default action
- C) Stops event
- D) Reload page

Answer: B

Q18.

```
div.addEventListener("click", () => console.log("div"));  
button.addEventListener("click", () => console.log("button"));
```

Click button → output?

- A) div
- B) button
- C) button → div
- D) div → button

Answer: C

Q19.

```
setTimeout(()=>console.log("Hi"), 0);  
console.log("Hello");
```

Output?

- A) Hi Hello
- B) Hello Hi
- C) Hi
- D) Error

Answer: B

Q20.

```
let url = location.href;  
console.log(typeof url);
```

- A) object
- B) string
- C) number
- D) undefined

Answer: B

Q21.

```
localStorage.setItem("x", 100);  
console.log(localStorage.getItem("x"));
```

- A) 100
- B) "100"
- C) undefined
- D) null

Answer: B

Q22.

```
sessionStorage.setItem("a", 5);  
sessionStorage.clear();  
console.log(sessionStorage.getItem("a"));
```

- A) 5
- B) "5"
- C) null
- D) undefined

Answer: C

Q23. Debouncing means:

- A) Execute function continuously
- B) Execute function only after delay when events stop
- C) Execute function every fixed interval
- D) Execute function randomly

Answer: B

Q24. Throttling means:

- A) Delay execution until events stop
- B) Execute function at most once in a given time interval
- C) Execute function multiple times instantly
- D) Stop function execution

Answer: B

Q25. Which is best for search input optimization?

- A) Throttling
- B) Debouncing
- C) setInterval
- D) forEach

Answer: B

Q26. Which is best for scroll/resize events?

- A) Debouncing
- B) Throttling
- C) JSON
- D) map()

Answer: B

Q27. Debounce Behavior

```
function debounce(fn, delay){  
  let timer;  
  return function(){  
    clearTimeout(timer);  
    timer = setTimeout(fn, delay);  
  }  
}
```

If user types continuously, function executes:

- A) On every keystroke
- B) Only once after user stops typing
- C) Every 1 second
- D) Twice

Answer: B

Q28. Throttle Behavior

```
function throttle(fn, limit){  
  let flag = true;  
  return function(){  
    if(flag){  
      fn();  
      flag = false;  
      setTimeout(()=> flag = true, limit);  
    }  
  }  
}
```

Function will execute:

- A) Every time event fires
- B) Only once ever
- C) At most once per limit time
- D) Never

Answer: C

Q29. Output Analysis (Debounce)

```
let fn = debounce(()=>console.log("Run"), 1000);  
fn();  
fn();  
fn();
```

What happens?

- A) Runs 3 times
- B) Runs once
- C) Runs twice
- D) Error

Answer: B

Q30. Output Analysis (Throttle)

```
let fn = throttle(()=>console.log("Run"), 1000);  
fn();  
fn();  
fn();
```

What happens immediately?

- A) Runs 3 times
- B) Runs once
- C) Runs twice
- D) Error

Answer: B

Q31. Identify Debounce

Which code represents debounce?

- A)
`setInterval(fn, 1000)`
- B)
`clearTimeout(timer);`
`timer = setTimeout(fn, delay);`
- C)
`fn();`
`fn();`
- D)
`map(fn)`

Answer: B

Q32. Identify Throttle

- A) `if(flag){`
`fn();`
`setTimeout(()=>flag=true, delay);`
`}`
- B) `setTimeout(fn, delay)`
- C) `clearTimeout(timer)`
- D) `filter(fn)`

Answer: A

Q33. Difference between debounce & throttle

- A) Both same
- B) Debounce delays, throttle limits frequency
- C) Throttle delays, debounce repeats
- D) Both stop execution

Answer: B

Q34. In debounce, if user never stops typing?

- A) Function runs continuously
- B) Function never runs
- C) Runs every second
- D) Error

Answer: B

Q35. In throttle, if events fire continuously?

- A) Function runs once
- B) Function runs at fixed intervals
- C) Function never runs
- D) Error

Answer: B

Q36.

```
let arr = [1,2,3];  
arr.pop();  
console.log(arr);
```

- A) [1,2,3]
- B) [1,2]
- C) [2,3]
- D) Error

Ans: B

Q37.

```
let arr = [1,2,3];  
arr.unshift(0);  
console.log(arr);
```

- A) [1,2,3,0]
- B) [0,1,2,3]
- C) [1,2,3]
- D) Error

Ans: B

Q38.

```
console.log([1,2,3].join("-"));
```

- A) "123"
- B) "1-2-3"
- C) [1-2-3]
- D) Error

Ans: B

Q39.

```
let arr = [1,2,3];  
delete arr[1];  
console.log(arr.length);
```

- A) 2
- B) 3
- C) 1
- D) Error

Ans: B

Q40.

```
console.log([1,2,3].indexOf(2));
```

- A) 1
- B) 2
- C) 0
- D) -1

Ans: A

Q41.

```
console.log([1,2,3].includes(5));
```

- A) true
- B) false
- C) undefined
- D) Error

Ans: B

Q42.

```
let arr = [1,2,3,4];  
console.log(arr.slice(1,3));
```

- A) [1,2]
- B) [2,3]
- C) [3,4]
- D) [2,3,4]

Ans: B

Q43.

```
let arr = [1,2,3];  
arr.splice(1,1);  
console.log(arr);
```

- A) [1,2,3]
- B) [1,3]
- C) [2,3]
- D) [1]

Ans: B

Q44.

```
console.log([1,2].concat([3,4]));
```

- A) [1,2,3,4]
- B) [3,4,1,2]
- C) [[1,2],[3,4]]
- D) Error

Ans: A

Q45.

```
console.log([1,[2,3]].flat());
```

- A) [1,2,3]
- B) [1,[2,3]]
- C) [[1,2,3]]
- D) Error

Ans: A

Q46.

```
[1,2,3].forEach(x => console.log(x));
```

- A) Returns array
- B) Prints values
- C) Returns number
- D) Error

Ans: B

Q47.

```
let result = [1,2,3].map(x => x+1);
```

Output?

- A) [1,2,3]
- B) [2,3,4]
- C) [1,3,5]
- D) Error

Ans: B

Q48.

```
let result = [1,2,3,4].filter(x => x%2==0);
```

- A) [1,3]
- B) [2,4]
- C) [4]
- D) []

Ans: B

Q49.

```
[1,2,3].reduce((a,b)=>a+b,0);
```

- A) 6
- B) 5
- C) 0
- D) Error

Ans: A

Q50.

```
[2,4,6].every(x => x%2==0);
```

- A) true
- B) false
- C) undefined
- D) Error

Ans: A

Q51.

```
[1,3,5].some(x => x%2==0);
```

- A) true
- B) false
- C) undefined
- D) Error

Ans: B

Q52.

```
let obj = {a:1,b:2};
```

```
console.log(Object.values(obj));
```

- A) [1,2]
- B) ["a","b"]
- C) [["a",1]]
- D) Error

Ans: A

Q53.

```
let obj = {x:10};
```

```
console.log(obj.x);
```

- A) 10
- B) undefined
- C) Error
- D) null

Ans: A

Q54.

```
let obj = {};
```

```
obj.name = "Ram";
```

```
console.log(obj);
```

- A) {}
- B) {name:"Ram"}
- C) Error
- D) null

Ans: B

Q55.

```
Object.entries({a:1});
```

- A) [["a",1]]
- B) [1]
- C) ["a"]
- D) Error

Ans: A

Q56.

```
let [a,b] = [1,2];  
console.log(b);
```

- A) 1
- B) 2
- C) undefined
- D) Error

Ans: B

Q57.

```
let [a=5] = [];  
Value of a?
```

- A) undefined
- B) 5
- C) null
- D) Error

Ans: B

Q58.

```
let {x} = {x:10};
```

- A) x=10
- B) x=undefined
- C) Error
- D) null

Ans: A

Q59.

```
let {a: b} = {a:5};  
console.log(b);
```

- A) a
- B) 5
- C) undefined
- D) Error

Ans: B

Q60.

```
JSON.stringify({a:1});
```

- A) Object
- B) String
- C) Array
- D) Error

Ans: B

Q61.

```
JSON.parse('{"a":2}');
```

- A) String
- B) Object
- C) Error
- D) Array

Ans: B

Q62.

querySelector() selects:

- A) All elements
- B) First match
- C) ID only
- D) Class only

Ans: B

Q63.

document.getElementById("id")

Returns:

- A) Single element
- B) List
- C) Array
- D) Error

Ans: A

Q64.

innerText is used to:

- A) Add HTML
- B) Get visible text
- C) Style element
- D) Delete node

Ans: B

Q65.

element.classList.add("active");

- A) Removes class
- B) Adds class
- C) Toggles class
- D) Deletes element

Ans: B

Q66.

addEventListener() is used for:

- A) Styling
- B) Event handling
- C) Storage
- D) JSON

Ans: B

Q67.

event.target

Refers to:

- A) Parent
- B) Clicked element
- C) Window
- D) Document

Ans: B

Q68.

stopPropagation() does:

- A) Stops default
- B) Stops bubbling
- C) Deletes event
- D) Logs event

Ans: B

Q69.

alert() shows:

- A) Console
- B) Popup
- C) Storage
- D) DOM

Ans: B

Q70.

setInterval(()=>console.log("Hi"),1000);

- A) Runs once
- B) Runs repeatedly
- C) Stops immediately
- D) Error

Ans: B

Q71.

location.href is used to:

- A) Reload page
- B) Get/set URL
- C) Close window
- D) Open popup

Ans: B

Q72.

localStorage data persists:

- A) Session
- B) Forever
- C) One minute
- D) Until reload

Ans: B

Q73.

localStorage.setItem("x","10");

Stores:

- A) Number
- B) String
- C) Object
- D) Boolean

Ans: B

Q74.

localStorage.clear();

- A) Deletes one item
- B) Deletes all
- C) Gets item
- D) Error

Ans: B

Q75.

`[1,2,3].find(x=>x>2);`

- A) 2
- B) 3
- C) true
- D) undefined

Ans: B

Q76.

`[1,2,3].findIndex(x=>x==2);`

- A) 2
- B) 1
- C) 0
- D) -1

Ans: B

Q77.

`Array.from("abc");`

- A) `["abc"]`
- B) `["a","b","c"]`
- C) `["a"]`
- D) Error

Ans: B

Q78.

`let arr=[1,2,3];
arr.copyWithin(1,0);
console.log(arr);`

- A) `[1,1,2]`
- B) `[1,2,3]`
- C) `[2,3,1]`
- D) Error

Ans: A

Q79.

`let arr=[1,2];
console.log(arr.toString());`

- A) `"1,2"`
- B) `[1,2]`
- C) `"12"`
- D) Error

Ans: A

Q80.

```
let arr=[1,2,3];  
console.log(arr.with(1,10));
```

- A) [1,10,3]
- B) [10,2,3]
- C) [1,2,3]
- D) Error

Ans: A

Q81.

```
let arr=[1,2,3];  
console.log(arr.at(1));
```

- A) 1
- B) 2
- C) 3
- D) undefined

Ans: B

Q82.

```
[1,2,3].reduceRight((a,b)=>a-b);
```

- A) -2
- B) 0
- C) 2
- D) Error

Q83.

```
let obj={};  
console.log(Object.keys(obj).length);
```

- A) 0
- B) 1
- C) undefined
- D) Error

Ans: A

Q84.

```
let arr=[1,2,3];  
console.log(arr.findLast(x=>x<3));
```

- A) 1
- B) 2
- C) 3
- D) undefined

Ans: B

Q85.

```
document.createElement("div");
```

- A) Deletes element
- B) Creates element
- C) Selects element
- D) Styles element

Ans: B